

 SPAMSHIELD AI

 EMAIL SECURITY

 DETECTION

 SMART FILTERING

SpamShield AI

Spam & Suspicious Email Detection System — Semester III

Year & Branch

2nd Year · CSE – AI & DS / AIDS

University

Pimpri Chinchwad University

By

Tanisha Pardhi



Python



Streamlit



NLP



Feature Extraction



Machine Learning



⚠️ THREAT ANALYSIS

🔥 SPAM DETECTION

📧 EMAIL SECURITY

Problem Statement

Automated email threat detection helps identify **spam** and suspicious messages faster, with less human error.

⚠️ Core Challenge

Manual review is slow and error-prone. The system must assess sender trust, link safety, intent, and language patterns to classify emails as **spam** or **legitimate**.

🔥 Common Threat Vectors

🎁 Promotions

Misleading offers and fake urgency.

👤 Phishing

Fraud links and impersonation.

🧠 Social Engineering

Pressure tactics and urgent language.

🔒 Data Harvesting

Requests for passwords, OTPs, and credentials.

⚠️ Detect threats early. 🔥 Reduce user risk. 📧 Keep inboxes safe. ✨ Stay alert, stay protected.



🔥 PROJECT OBJECTIVES

★ VIBRANT TECH

🛡️ SPAMSHIELD AI

What SpamShield AI Sets Out to Achieve

Bright, fast, and intelligent email protection with a modern, high-energy visual style.



Automated Classification 🔄

Classify every incoming email as **Spam** or **Legitimate** without manual intervention, using a deterministic feature-weighted scoring pipeline.



Text Analysis & NLP 🔍

Analyse textual patterns — sentence structure, word distribution, linguistic cues — to surface anomalies indicative of malicious intent.



Feature Extraction ✅

Systematically identify suspicious keywords, embedded URLs, excessive capitalisation, punctuation density, and currency symbols.



Confidence Estimation 🛡️

Output a normalised confidence score (via sigmoid function) so users understand the system's certainty level for each classification.



Interactive Interface ✅

Provide a clean, accessible Streamlit-based web application with real-time scanning, result display, and session-based scan history.



Future ML Model 🔮

Train and evaluate a Multinomial Naive Bayes model on a real labelled dataset for quantitative performance benchmarking in subsequent reviews.

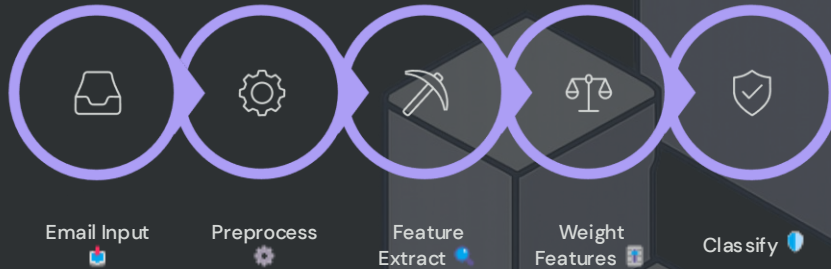


▲ Designed to feel **energetic**, **modern**, and **visually striking** while keeping the content readable.



Processing Pipeline & System Boundaries ⚡

👤 End-to-End Processing Pipeline



The pipeline transforms raw email text into a structured classification decision with an associated confidence score, using a feature-weighted scoring engine and sigmoid normalisation.

⚙️ Current Scope

→ Email Text Analysis

Full body text accepted as input; tokenised and processed for feature detection.

→ Keyword & Pattern Detection

Spam and legitimate keyword banks; suspicious URL matching via regex.

→ Confidence Estimation

Weighted feature score normalised via sigmoid to a 0-100% confidence output.

→ 🌐 Interactive Web Interface

Streamlit UI with real-time output and session-level scan history.

⚠️ Current limitations: no header analysis, no SPF/DKIM/DMARC verification, no attachment scanning.



Literature Review

Comparative Analysis of [Spam Detection Approaches](#)

Approach	Method	Advantages	Limitations
Rule-Based	Keyword matching & blacklists	Lightweight, fully explainable, no training data required	Easily bypassed by paraphrasing; brittle to new patterns
Naïve Bayes	Probabilistic classification over word frequencies	Simple to implement, effective for text, interpretable	Requires clean labelled dataset; assumes feature independence
TF-IDF + ML	Feature weighting with supervised ML algorithms	Captures term importance, strong baseline accuracy	Requires preprocessing pipeline and training infrastructure
Deep Learning	Neural networks (LSTM, BERT, transformers)	Learns complex, contextual patterns; state-of-the-art results	Requires large data volumes, significant compute resources

- 📌 **Research Gap addressed by this project:** SpamShield AI begins with a lightweight, explainable feature-weighted baseline — deliberately positioned at the rule-based tier — and plans to extend into a trained Multinomial Naive Bayes model. This staged approach allows direct, measurable comparison between the two paradigms within the same codebase.



Proposed Technical Methodology

Colorful feature-weighted spam detection with a clear path to machine learning.

Processing Pipeline — Step by Step

01

Email Input

Raw email body text via Streamlit text area.

02

Text Preprocessing

Lowercasing, punctuation normalization, whitespace handling.

03

Tokenisation

Text split into tokens for analysis.

04

Feature Extraction

Keyword, URL, uppercase, punctuation, and currency signals.

05

Spam / Ham Scoring

Features add positive or negative weight to the score.

06

Sigmoid Normalisation

Score mapped to a 0–1 confidence probability.

07

Final Classification

Threshold applied: above → SPAM, below → LEGITIMATE.

Feature Dictionary — Extraction Categories

Spam Keywords

click here · urgent · verify account · limited offer

Legitimate Keywords

meeting · project · deadline · attached · review

Textual Signals

Uppercase ratio · exclamations · word count · sentence length

Structural Signals

Suspicious URLs · currency symbols (\$, £, €)

Current Base line: Feature-Weighted Classification
Planned Next Phase: Multinomial Naive Bayes on labelled dataset



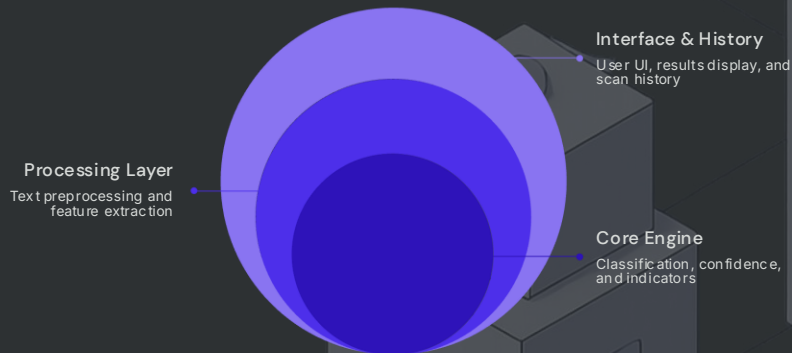
VIBRANT TECH STACK

MODERN ARCHITECTURE

System Architecture

System Architecture

Full System Architecture Flow



The architecture is modular and single-process: the Streamlit UI drives the Python backend directly, with each layer passing structured data to the next. Scan history is maintained in session state for within-session review.

Input

Streamlit text area for email body

Processing

Preprocessing, tokenisation, and feature scoring

Output

Spam / ham result, confidence, and history

Technology Stack



Python

Core classification logic



Streamlit

Interactive UI and session history



Regex

URL and signal detection



NLP

Text normalisation and tokenisation



ML — Naive Bayes

Planned Phase 2 classifier

Bright accent colors help separate the UI, processing, and output layers at a glance.



Implementation

A fully working prototype has been implemented in Python and deployed via Streamlit. The current build includes the following core features:



✨ Email Input

Paste raw email content for instant analysis.



🚀 Classification

Outputs spam or legitimate labels from scoring.



💡 Confidence Score

Shows a sigmoid-normalized probability score.



🎨 Keyword Highlighting

Highlights important spam and legitimate signals.



🔗 URL Detection

Flags suspicious links using regex patterns.



🕒 Scan History

Tracks scans within the current session.



Screenshot Placeholder

Insert the final Streamlit prototype screenshot here. Keep this area compact so it remains clearly visible without overflowing the card.



Test Results

The following test cases demonstrate current system behaviour across three distinct email categories. These are qualitative demonstrations of feature detection — quantitative metrics will be computed in Phase 2 using a labelled dataset.

TEST CASE 1

Promotional Email → SPAM

*"Click here NOW! Limited offer – 50% OFF!
Verify your account immediately!"*

Detected indicators: spam keywords,
uppercase emphasis, exclamation density,
urgency language

Classification: **SPAM** — high confidence

TEST CASE 2

Project Meeting Email → LEGITIMATE

*"Hi, please review the project proposal for
the meeting on Friday at 2 PM."*

Detected indicators: legitimate keywords,
normal capitalisation, professional
structure, no suspicious URLs

Classification: **LEGITIMATE** — high
confidence

TEST CASE 3

Suspicious Account Verification → SPAM / SUSPICIOUS

*"Verify your account now:
[http://suspicious-
link.com/verify?id=12345](http://suspicious-link.com/verify?id=12345)"*

Detected indicators: suspicious URL,
verification request, urgency language,
external domain

Classification: **SPAM / SUSPICIOUS** —
elevated confidence



Screenshot Placeholder: Insert Streamlit output screenshots for each test case here before final submission.



Roadmap & Conclusion

ROADMAP

NEXT STEPS

GROWTH

THANK YOU

📋 ✨ Vibrant roadmap ahead: from prototype to a scalable, model-driven SpamShield AI.

Phase 2 — ML Model Development

1 Labelled Data

Use UCI spam or Enron corpus.

2 Naive Bayes

Train and evaluate the classifier.

3 TF-IDF Features

Upgrade from keyword lists.

4 Benchmark Models

Compare Logistic Regression, SVM, and Random Forest.

Phase 3 — Advanced Capabilities

→ URL Detection

Detect phishing links.

→ Header Analysis

Check SPF, DKIM, and DMARC.

→ Attachment Scan

Inspect file metadata and signatures.

→ Cloud Deploy

Launch on a scalable platform.

Conclusion: SpamShield AI is a clear, lightweight, and explainable spam classifier built as a working end-to-end prototype. It combines preprocessing, feature extraction, scoring, and confidence estimation into one interactive Streamlit app. The next phase will replace heuristics with trained models and stronger evaluation, turning this prototype into a more accurate, robust, and scalable system.



Thank You — Questions Welcome

